

Curso de desenvolvemento de aplicacións con tecnoloxías web

Módulo formativo 3: Implantación de aplicacións web en contornos de internet, intranet e extranet

Tema 4: Desenvolvemento e despregamento de aplicacións web

Modelos básicos de desenvolvemento de aplicacións web. O modelo vista-controlador (MVC)

Ferramentas de desenvolvemento web de uso común

Políticas de desenvolvemento e probas de aplicacións web

Organización de recursos nunha aplicación web

Modelos básicos de desenvolvemento de aplicacións web. O modelo vista-controlador (MVC)

Modelo clásico de desenvolvemento web

O modelo tradicional baséase en:

- HTML estático: páxinas servidas tal cal ao navegador.
- Separación mínima entre lóxica, estrutura e presentación.
- Escasa reutilización de código.

Modelo Cliente-Servidor

- O cliente (navegador) realiza peticións.
- O servidor responde con contido (HTML, JSON, etc.).
- Exemplo: unha aplicación en JavaScript que consume unha API REST en Node.js ou PHP.

Modelo dunha soa páxina (SPA)

- Toda a aplicación cárgase unha soa vez.
- A navegación e as actualizacións realízanse de forma dinámica dende o navegador.
- Tecnoloxías: React, Angular, Vue.js.

Modelo Vista-Controlador (MVC)

- **Separación de responsabilidades:**
 - Modelo: xestión de datos e lóxica de negocio (ex. clases que xestionan unha base de datos).
 - Vista: interface gráfica para o usuario (HTML, CSS).
 - Controlador: intermediario que procesa peticións, actualiza o modelo e selecciona a vista.
- **Vantaxes:**
 - Código máis limpo e mantible.
 - Facilita o traballo en equipo.
 - Reutilización de compoñentes.

Exemplo sinxelo en PHP (MVC):

```
// modelo.php
class Usuario {
    public function obterUsuarios() {
        return ["Xoán", "María", "Lucía"];
    }
}

// controlador.php
require 'modelo.php';
$modelo = new Usuario();
$usuarios = $modelo->obterUsuarios();
include 'vista.php';

// vista.php
foreach ($usuarios as $usuario) {
    echo "<p>$usuario</p>";
}
```

Ferramentas de desenvolvemento web de uso común

Características das ferramentas de desenvolvemento web

As ferramentas de desenvolvemento web axudan a crear, editar, probar e despregar aplicacións web. Algunhas das súas características comúns son:

- **Entornos integrados de desenvolvemento (IDE):** permiten escribir código con axuda de coloreado, autocompletado, integración con control de versións, etc.
Exemplos: Visual Studio Code, PhpStorm, Eclipse, NetBeans.
- **Xestores de dependencias:** permiten instalar e actualizar librarías ou frameworks dun proxecto.
Exemplos: npm (JavaScript), Composer (PHP), pip (Python).
- **Frameworks:** estruturas que facilitan a organización do código e a aplicación de boas prácticas.
Exemplos: Laravel (PHP), Spring Boot (Java), Express (Node.js), Django (Python).
- **Ferramentas de control de versións:** permiten rexistrar cambios no código e traballar en equipo.
Exemplo principal: Git (xunto con plataformas como GitHub, GitLab ou Bitbucket).
- **Preprocesadores:** ferramentas que permiten escribir código en linguaxes que logo se compilan ou transforman.
Exemplos: SASS (CSS), TypeScript (JavaScript).
- **Contedores e virtualización:** permiten simular contornos de produción nunha máquina local.
Exemplo principal: Docker.
- **Sistemas de xestión de bases de datos:** ferramentas para crear e xestionar bases de datos relacionais ou non relacionais.
Exemplos: MySQL, PostgreSQL, MongoDB, SQLite.
- **Cientes e ferramentas para probar APIs:** permiten simular peticións HTTP e probar servizos web.
Exemplos: Postman, Insomnia, cURL.

Comparativa básica entre ferramentas

Ferramenta	Uso principal	Vantaxes	Inconvenientes
Visual Studio Code	Edición e programación	Lixeiro, moitos plugins	Requiere configuración inicial
PhpStorm	IDE para PHP	Completo, integración avanzada	De pago
Git + GitHub	Control de versións e colaboración	Seguimento, ramas, historial	Requiere curva de aprendizaxe
Docker	Virtualización de contornos	Aislamento, reproducibilidade	Uso de recursos elevado
Postman	Probas de APIs	Interface gráfica amigable	Algunhas funcións son de pago

Políticas de desenvolvemento e probas de aplicacións web

As políticas de desenvolvemento e probas establecen normas e procedementos que aseguran que unha aplicación web se desenvolve e valida correctamente antes da súa publicación definitiva.

- **Entorno de desenvolvemento**

É o contorno onde se crea e programa a aplicación. As súas características principais son:

- Dispoñible localmente para os programadores.
- Inclúe ferramentas como IDEs, servidores locais, xestores de dependencias e bases de datos de proba.
- Permite facer cambios de maneira rápida e realizar probas unitarias e de integración.

Exemplo: Un programador traballa localmente cun servidor Apache e unha base de datos MySQL instalados en Docker.

- **Entorno de preproducción ou probas**

É unha réplica do entorno de produción, onde se fan probas antes da posta en marcha definitiva. Úsase para:

- Probas funcionais e de rendemento.
- Validación por parte de usuarios ou clientes.
- Probas de integración co resto de servizos.

Exemplo: Unha empresa crea un subdominio como test.exemplo.com onde se cargan as versións candidatas para revisión.

- **Entorno de produción**

É o contorno onde a aplicación está dispoñible para os usuarios finais. As súas características son:

- Estabilidade e rendemento son prioritarios.
- Os cambios deben pasar previamente polos contornos de desenvolvemento e probas.
- Inclúe medidas de seguridade, monitorización e copias de seguridade.

Boas prácticas:

- Automatización do despregue (CI/CD).
- Rexistro de erros e supervisión do sistema.
- Plans de recuperación ante fallos.

Organización de recursos nunha aplicación web

Nunha aplicación web, os recursos organízanse de maneira estruturada para facilitar o desenvolvemento, mantemento, escalabilidade e seguridade. Estes recursos inclúen arquivos que forman parte do código fonte, configuracións, elementos visuais e bibliotecas de terceiros.

Programas

Son os ficheiros que conteñen o código da lóxica da aplicación. Inclúen scripts ou ficheiros escritos en linguaxes como PHP, Python, JavaScript (Node.js), Java, etc.

- **Pódense organizar en cartafoles segundo a súa función:** controladores, modelos, servizos, etc.
- **Exemplo:** controllers/, services/, routes/.

Follas de estilo

Conteñen as regras CSS que definen a presentación visual da aplicación.

- Fanse chamadas dende as páxinas HTML ou mediante preprocesadores como SCSS ou LESS.
- Exemplo: styles/main.css, assets/css/.

Ficheiros de configuración

Definen parámetros do funcionamento da aplicación segundo o entorno (desenvolvemento, produción...).

- Inclúen rutas, conexións a bases de datos, variables de entorno, etc.
- Exemplo: .env, config/app.php, settings.json.

Imaxes

Arquivos gráficos utilizados no deseño da interface ou no contido visual.

- Organízanse por tipo ou función (icons, banners, logos...).
- Exemplo: assets/images/logo.png, img/icons/.

Documentos

Arquivos que se poden descargar ou consultar na web, como PDFs, informes, manuais, etc.

- Exemplo: docs/guía_usuario.pdf.

Bibliotecas de compoñentes (librarías)

Código reutilizable que engade funcionalidades á aplicación.

- Poden ser bibliotecas JS como jQuery ou frameworks como React ou Vue.
- Descárganse e xestiónanse con ferramentas como npm, composer, etc.
- Exemplo: node_modules/, vendor/.

Outros arquivos

Inclúen outros elementos necesarios como fontes tipográficas, vídeos, sons, etc.

- Organízanse en cartafolios específicos segundo o seu tipo ou uso.
- Exemplo: fonts/, media/videos/, audio/.

Seguridade nunha aplicación web

A seguridade nas aplicacións web é fundamental para protexer os datos dos usuarios e garantir o correcto funcionamento do sistema fronte a ataques ou accesos non autorizados. A continuación descríbense os principais conceptos e técnicas empregadas neste eido.

Niveis de seguridade. Estándares

A seguridade pode abordarse en distintos niveis:

- **Seguridade no servidor:** configuración do servidor web e da base de datos, actualizacións, peche de portos innecesarios, etc.
- **Seguridade na aplicación:** validación de formularios, xestión segura de sesións e contrasinais, control de accesos.
- **Seguridade na transmisión:** uso de HTTPS (SSL/TLS) para cifrar a comunicación.
- **Seguridade no cliente:** protección fronte a scripts maliciosos (XSS), control de cookies, etc.

Estándares recoñecidos como **OWASP** (Open Web Application Security Project) proporcionan guías e boas prácticas para prever vulnerabilidades comúns como:

- Inxección SQL
- Cross-site scripting (XSS)
- Falsificación de peticións entre sitios (CSRF)
- Mal uso da autenticación ou sesións

Conceptos e técnicas de identificación, autenticación e autorización ou control de acceso

- **Identificación:** recoñecemento do usuario, normalmente mediante un nome de usuario ou correo electrónico.
- **Autenticación:** verificación da identidade, normalmente con contrasinal, pero tamén con métodos como códigos de verificación, tokens ou certificados dixitais.

- **Autorización:** determinación dos permisos que ten un usuario unha vez autenticado. Por exemplo, un usuario administrador poderá editar contido mentres outro só poderá visualizar.

Sistemas habituais:

- Roles de usuario (ex. admin, editor, lector)
- Listas de control de acceso (ACL)

Identificación e autenticación avanzada. Certificados dixitais

Métodos avanzados para reforzar a autenticación:

- **Dobre factor (2FA):** uso dun segundo elemento de verificación (SMS, app, correo).
- **Certificados dixitais:** empregados para autenticación de alto nivel (ex. sinatura dixital, acceso seguro a plataformas gobernamentais).
- **Tokens de acceso (JWT, OAuth):** utilizados para sesións seguras e sistemas distribuídos.

Concepto de sesión. Conservación de sesións

Unha sesión permite manter o estado dunha aplicación web mentres o usuario navega por distintas páxinas.

- As sesións poden gardarse no servidor, no navegador (cookies, localStorage), ou mediante tokens.
- Deben conter información mínima e protexerse contra ataques de secuestro de sesión (session hijacking).

Sistemas de uso común para a conservación das sesións en aplicacións web. Single Sign-on e Single Sign-out

- **Conservación tradicional:** mediante cookies de sesión ou almacenamento local.
- **Single Sign-On (SSO):** permite que o usuario acceda a varias aplicacións cunha única autenticación. Ex. Google, Facebook login.
- **Single Sign-Out:** pecha a sesión en todas as aplicacións asociadas cando se pecha nunha soa.

Estes sistemas requiren protocolos como OAuth2, OpenID Connect e unha xestión adecuada dos tokens de sesión.

Despregue de aplicacións web

O despregue é a etapa final do desenvolvemento dunha aplicación web, na que se publica e pon en funcionamento nun servidor accesible para os usuarios finais. É un proceso clave que require planificación, seguridade e verificación.

Características do proceso de despregue

O proceso de despregue debe garantir que a aplicación:

- Funcione correctamente no entorno de produción.
- Estea configurada axeitadamente (conexións a bases de datos, rutas, dominios...).
- Estea protexida contra accesos non autorizados.
- Permita unha monitorización e mantemento eficiente.

Un despregue pode ser:

- **Manual:** feito paso a paso por unha persoa (máis propenso a erros).
- **Automatizado:** mediante ferramentas que permiten reproducir o proceso de forma segura e consistente (ex. CI/CD).

Tamén é importante definir se será:

- Despregue completo (full deploy): substitúe toda a aplicación.
- Despregue incremental (hotfixes ou actualizacións parciais): só os cambios.

Definición do proceso de despregue de aplicacións web. Verificación

O proceso típico de despregue inclúe as seguintes fases:

1. Preparación do entorno:

- Crear ou configurar o servidor (Apache, Nginx, Tomcat...).

- Instalar dependencias (linguaxes, motores de bases de datos, bibliotecas...).
- Configurar dominios, certificados SSL e variables de entorno.

2. Despregue do código:

- Subida mediante FTP, SSH, Git, ou sistemas de integración continua (Jenkins, GitLab CI, GitHub Actions...).
- Instalación e compilación (npm, Maven, Composer, etc.).
- Migracións ou actualizacións da base de datos, se fose preciso.

3. Configuración e posta en marcha:

- Establecer parámetros segundo o entorno (producción, pre-producción...).
- Configurar logs e monitorización (logrotate, Prometheus, etc.).
- Reinicio dos servizos necesarios (web server, aplicación, cache...).

4. Verificación do despregue:

- Probas automáticas e/ou manuais para confirmar o correcto funcionamento.
- Comprobación do rendemento e da seguridade.
- Validación da conectividade con outros sistemas ou servizos.

Un bo despregue require unha planificación previa e, idealmente, debe realizarse primeiro nun entorno de **pre-producción**.